

ALEGERE TEMA - ALGORITMI PARALELI SI DISTRIBUITI

Procesarea Paralela a Imaginilor (Image Processing)

Daria Giotina

CR3.3 A

1. PREZENTAREA TEMEI

Proiectul de fata are ca tema procesarea de imagini. Acest domeniu presupune aplicarea unor operatii matematice pe matricele de pixeli care formeaza o imagine digitala. Printre cele mai comune operatii se numara filtrarea (blur, detectia muchiilor) si modificarea culorilor.

Aceste operatii sunt foarte potrivite pentru paralelizare. Motivul este ca, in mare parte, pixelii pot fi procesati independent unii de altii. Astfel, prin impartirea imaginii in mai multe sectiuni si procesarea lor simultana folosind mai multe thread-uri, timpul total de executie poate fi redus semnificativ comparativ cu o implementare secventiala clasica.

2. OBIECTIVELE PROIECTULUI

Obiectivele principale ale acestui proiect sunt:

- a) Implementarea unei solutii secventiale care aplica o serie de filtre (Sobel, Gaussian Blur, Negativ) pe o imagine.
- b) Dezvoltarea unei solutii paralele pentru aceleasi filtre, impartind imaginea in regiuni care sa fie procesate simultan de mai multe fire de executie.
- c) Masurarea si compararea timpilor de executie intre cele doua abordari pe imagini de dimensiuni diferite.
- d) Calcularea speedup-ului si a eficientei algoritmului paralel in functie de numarul de thread-uri folosite.

3. METODE SI ALGORITMI UTILIZATI

3.1. Filtrul Sobel (Detectia muchiilor)

Acest filtru este folosit pentru a determina contururile obiectelor dintr-o imagine. Utilizeaza doua kernel-uri de convolutie (Gx si Gy) pentru a calcula gradientul de intensitate pe orizontala si verticala.

Kernel Gx: Kernel Gy:

-1 0 +1 -1 -2 -1

-2 0 +2	0 0 0
-1 0 +1	+1 +2 +1

Magnitudine: $G = \sqrt{G_x^2 + G_y^2} + 1 + 2 + 1$

Deoarece calculul se face independent pentru fiecare pixel in parte, operatia este usor de paralelizat.

3.2. Gaussian Blur (Netezirea imaginii)

Acest filtru reduce zgomotul din imagine facand o medie intre un pixel si vecinii sai, ponderata de o distributie gaussiana. Voi folosi un kernel standard de 3x3 (impartit la 16):

1 2 1

2 4 2

1 2 1

3.3. Filtrul Negativ

Este o transformare simpla in care fiecare valoare a canalelor de culoare (R, G, B) este inversata: $\text{valoare_noua} = 255 - \text{valoare_initiala}$. Aici procesarea fiecarui pixel este complet independenta.

3.4. Abordarea pentru paralelizare

Pentru paralelizare voi imparti imaginea pe linii. Fiecare thread va primi un interval de linii pe care sa il proceseze. De exemplu, pentru o imagine cu H linii si T thread-uri:

- Thread 0 : liniile de la 0 la $H/T - 1$
- Thread 1 : liniile de la H/T la $2*H/T - 1$
- ...
- Thread T-1: liniile de la $(T-1)*H/T$ la $H - 1$

4. TEHNOLOGII SI LIMBAJ DE PROGRAMARE

- **Limbaj:** Java (JDK 19+)
- **Motivele alegerii:** Java ofera un suport stabil si usor de folosit pentru lucrul cu thread-uri (clasa `java.lang.Thread`). De asemenea, contine biblioteci standard pentru manipularea imaginilor (`BufferedImage`, `ImageIO`), ceea ce simplifica citirea si scrierea fisierelor.

- **Mecanisme de paralelizare:** Voi crea manual thread-urile, le voi aloca segmente de linii din imagine, iar la final voi folosi metoda `join()` pentru a ma asigura ca toate thread-urile si-au terminat executia inainte de salvare.
- **Formate suportate:** JPEG si PNG.
- **Mediu de dezvoltare:** Visual Studio Code cu extensiile pentru Java.

5. STRUCTURA PROIECTULUI

Proiectul va fi organizat astfel:

- `src/Main.java` - Punctul de intrare in program, responsabil de testare si afisarea timpilor.
- `src/ImageUtils.java` - Clasa cu metode utilitare pentru citirea si salvarea imaginilor.
- `src/SequentialImageProcessor.java` - Implementarea secventiala a filtrelor.
- `src/ParallelImageProcessor.java` - Implementarea paralelizata a acelorasi filtre.
- `output/` - Directorul in care vor fi salvate imaginile rezultate.

Ambele clase de procesare vor avea metode specifice pentru fiecare filtru in parte si vor returna obiectul `BufferedImage` modificat.

6. PLANUL DE EVALUARE A PERFORMANTEI

Pentru a analiza castigul adus de paralelizare, voi masura:

- Timpul de executie secvential (T_{seq})
- Timpul de executie paralel (T_{par}) pentru 2, 4, 8 si 16 thread-uri.

Ulterior voi calcula:

- Speedup $S(T) = T_{seq} / T_{par}(T)$
- Eficiență $E(T) = S(T) / T$

Testele vor fi rulate pe imagini cu rezolutii diferite (512x512, 1024x768, 1920x1080) pentru a observa influenta dimensiunii datelor asupra performantei. Toate rezultatele vor fi centralizate sub forma unui tabel comparativ.

7. REZULTATE ASTEPTATE

Pentru filtrul Negativ, ma astept la un speedup aproape liniar odata cu cresterea numarului de thread-uri, deoarece nu exista dependente intre pixeli. Pentru filtrele de convolutie (Sobel si Gaussian Blur), speedup-ul va fi probabil sub-liniar din cauza overhead-ului introdus de crearea thread-urilor si de procesarea pixelilor de la bordurile segmentelor.

Pe un procesor quad-core, estimez ca voi obtine un speedup de aproximativ $3x - 3.5x$ pentru 4 thread-uri, in special pe imaginile de dimensiuni mari.